

# **LAPORAN TUGAS BESAR**

## **Rekonstruksi Kasus GitLab Password Reset Vulnerability**



**Dipersiapkan oleh:**

**I Gede Agus Yoga Candra Wijaya / 230712518**

**Muhammad Tafarrel Dhafa Athaya / 230712595**

**Marcellinus Sendy Widiatmoko / 230712410**

**Gabriela Mitchell Hendri / 230712391**

**Godwin Xavier Kanna / 230712392**

**Dwi Miftah Estiawan / 230712378**

**Yuvens Aryaka Dewarandryasta / 230712403**

**Kadmiel Kislew Stevano Karenda / 230712490**

**Felecianus Ian Rajendra Sancha Tany / 230712446**

**Beda Arya Wimala / 230712345**

**Kelvin / 230712436**

**PROGRAM STUDI INFORMATIKA**

**FAKULTAS TEKNOLOGI INDUSTRI**

**UNIVERSITAS ATMA JAYA YOGYAKARTA**

# DAFTAR PUSTAKA

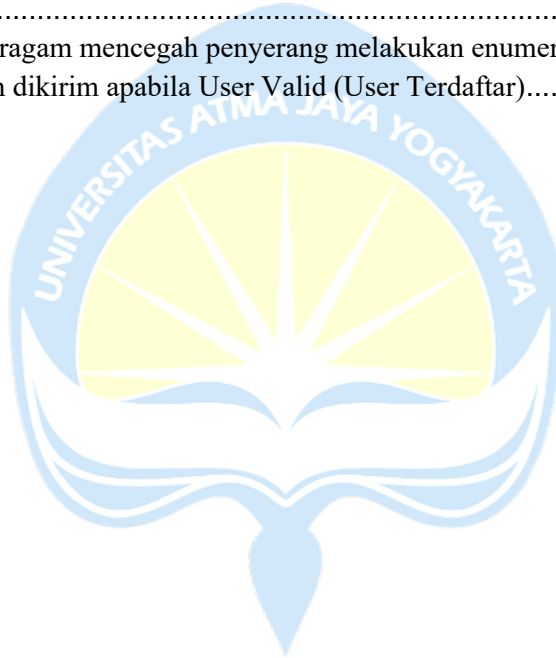
|                                                                                |                   |
|--------------------------------------------------------------------------------|-------------------|
| <b>DAFTAR PUSTAKA .....</b>                                                    | <b><i>i</i></b>   |
| <b>DAFTAR GAMBAR .....</b>                                                     | <b><i>iii</i></b> |
| <b>DAFTAR TABLE .....</b>                                                      | <b><i>iv</i></b>  |
| <b>1 PENDAHULUAN .....</b>                                                     | <b>1</b>          |
| 1.1 Ringkasan Kasus.....                                                       | 1                 |
| 1.2 Kasus yang Dipilih .....                                                   | 2                 |
| 1.3 Tujuan Tugas.....                                                          | 3                 |
| <b>2 Root Cause Reconstruction &amp; Invariant Design.....</b>                 | <b>4</b>          |
| 2.1 Model Domain Awal (Sebelum Perbaikan).....                                 | 4                 |
| 2.1.1 State Diagram Berbahaya .....                                            | 6                 |
| 2.1.2 State Berbahaya yang Dapat Direpresentasikan .....                       | 8                 |
| 2.2 Analisis Root Cause .....                                                  | 9                 |
| 2.2.1 Hubungan ke Konsep Valid State Space .....                               | 10                |
| 2.3 Perbaikan dengan Invariant/Domain Primitive .....                          | 11                |
| 2.3.1 Diagram State Setelah Perbaikan.....                                     | 11                |
| 2.3.2 Aturan Domain .....                                                      | 12                |
| 2.4 Perancangan Perbaikan dengan Invariant .....                               | 14                |
| 2.4.1 Invariant 1: Validasi Tipe Data Parameter Email Secara Eksplisit .....   | 14                |
| 2.4.2 Invariant 2: Email Tujuan Harus Terverifikasi Milik Akun .....           | 15                |
| 2.4.3 Invariant 3: Token Reset Terikat pada Penerima Email yang Spesifik ..... | 16                |
| 2.4.4 Invariant 4: Rate Limiting dan Monitoring.....                           | 16                |
| <b>3 State Machine &amp; Temporal Invariant Project.....</b>                   | <b>18</b>         |
| 3.1 Diagram State dan Transisi State .....                                     | 18                |
| 3.2 Enum State .....                                                           | 19                |
| 3.3 Temporal Invariant.....                                                    | 20                |
| 3.3.1 Definisi Struktur Class Utama.....                                       | 20                |
| 3.3.2 Tambahkan Initial State di Konstruktor.....                              | 21                |
| 3.3.3 Tambahkan Pengecekan State di dispatch() .....                           | 22                |
| 3.3.4 Tambahkan Pengecekan State di validate() .....                           | 22                |
| 3.3.5 Tambahkan Pengecekan State di consume() .....                            | 23                |
| 3.3.6 Tambahkan Pengecekan State Expired.....                                  | 23                |
| 3.3.7 Integrasi Service Layer .....                                            | 24                |
| <b>4 Mini-Project: Forgot Password: Secure Design Review.....</b>              | <b>26</b>         |

|                             |                                                             |           |
|-----------------------------|-------------------------------------------------------------|-----------|
| <b>4.1</b>                  | <b>Alur Kerja Sistem .....</b>                              | <b>26</b> |
| <b>4.2</b>                  | <b>Vulnerable System.....</b>                               | <b>28</b> |
| 4.2.1                       | Kode Sistem .....                                           | 28        |
| <b>4.3</b>                  | <b>Secure System.....</b>                                   | <b>32</b> |
| 4.3.1                       | Tambahkan Domain Primitive.....                             | 32        |
| 4.3.2                       | Tambahkan Form Request Validation .....                     | 33        |
| 4.3.3                       | Tambahkan Enum State .....                                  | 33        |
| 4.3.4                       | Tambahkan State Machine Domain dan Temporal Invariant ..... | 34        |
| <b>4.4</b>                  | <b>Kode Final Secure Design .....</b>                       | <b>38</b> |
| <b>4.5</b>                  | <b>Simulasi Serangan pada Secure System .....</b>           | <b>40</b> |
| <b>DAFTAR PUSTAKA .....</b> |                                                             | <b>42</b> |



## DAFTAR GAMBAR

|                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------|----|
| Gambar 2.1 Diagram State Berbahaya .....                                                                           | 7  |
| Gambar 2.2 Diagram State Perbaikan .....                                                                           | 12 |
| Gambar 3.1 Diagram State dan Transisi State.....                                                                   | 19 |
| Gambar 4.1 Alur bisnis system lupa password.....                                                                   | 27 |
| Gambar 4.2 Tampilan antarmuka website lupa password yang rentan.....                                               | 28 |
| Gambar 4.3 Penyerang pergi ke endpoint GET /forgot-password untuk mengambil token csrf.....                        | 31 |
| Gambar 4.4 Penyerang mengirim payload array pada field email dan token csrf untuk mengelabui keamanan laravel..... | 31 |
| Gambar 4.5 Token yang dikirim ke email korban .....                                                                | 31 |
| Gambar 4.6 Token yang dikirim ke email penyerang .....                                                             | 31 |
| Gambar 4.7 Tampilan antarmuka website lupa password yang secure.....                                               | 38 |
| Gambar 4.8 Penyerang mengirim payload array ditolak HTTP 422 oleh Form Request .....                               | 40 |
| Gambar 4.9 Respons seragam mencegah penyerang melakukan enumerasi akun (User Tidak Terdaftar) .....                | 40 |
| Gambar 4.10 Respons seragam mencegah penyerang melakukan enumerasi akun (User Terdaftar) ..                        | 41 |
| Gambar 4.11 Token akan dikirim apabila User Valid (User Terdaftar).....                                            | 41 |



## DAFTAR TABLE

|                                                    |    |
|----------------------------------------------------|----|
| Tabel 1.1 Ringkasan Kasus.....                     | 1  |
| Tabel 1.2 Tujuan Laporan .....                     | 3  |
| Tabel 2.1 State yang Dapat Direpresentasikan ..... | 8  |
| Tabel 2.2 Aturan Domain .....                      | 13 |



# 1 PENDAHULUAN

Keamanan perangkat lunak merupakan aspek kritis dalam pengembangan sistem modern, khususnya pada platform yang digunakan secara luas oleh komunitas pengembang di seluruh dunia. Salah satu bentuk ancaman yang paling berbahaya adalah kerentanan pada mekanisme autentikasi, karena celah pada lapisan ini dapat memberikan akses penuh kepada pihak yang tidak berwenang tanpa memerlukan interaksi dari korban. Laporan ini menganalisis secara mendalam kerentanan CVE-2023-7028 yang ditemukan pada platform GitLab, sebuah celah kritis pada fitur *password reset* yang memungkinkan terjadinya *account takeover* dengan skor CVSS 10.0.

Analisis ini mencakup identifikasi akar penyebab kerentanan, pemodelan *state* berbahaya yang dapat direpresentasikan oleh sistem, serta perancangan perbaikan berbasis *invariant* dan *domain primitive* untuk memastikan kondisi berbahaya tersebut tidak dapat terjadi secara struktural. Pendekatan yang digunakan mengacu pada prinsip *secure by design*, di mana sistem dirancang sedemikian rupa sehingga kondisi yang tidak valid secara domain tidak mungkin direpresentasikan, bukan hanya dicegah melalui pemeriksaan *runtime*.

## 1.1 Ringkasan Kasus

Bagian ini menyajikan gambaran umum mengenai insiden kerentanan CVE-2023-7028 pada GitLab secara ringkas dan terstruktur. Data yang disajikan mencakup identitas perusahaan yang terdampak, periode kerentanan aktif, cakupan korban, jenis data yang terekspos, serta cara kerentanan tersebut pertama kali ditemukan dan dilaporkan. Ringkasan lengkap kasus ini dapat dilihat pada Tabel 1.1 berikut.

Tabel 1.1 Ringkasan Kasus

| Atribut          | Detail                                                                                                                                                                                 |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nama Perusahaan  | GitLab                                                                                                                                                                                 |
| Tahun            | 2023 (Ditemukan akhir 2023, dirilis patch resmi pada Januari 2024)                                                                                                                     |
| Lokasi           | Global                                                                                                                                                                                 |
| Korban           | Seluruh pengguna GitLab Community Edition (CE) dan Enterprise Edition (EE) pada versi yang terdampak (mulai versi 16.1 hingga sebelum patch)                                           |
| Data Terekposisi | Akses penuh ke akun pengguna (Account Takeover), yang mencakup source code perusahaan, API keys, CI/CD secrets, dan data pribadi                                                       |
| Kerugian         | Skor CVSS: 10.0 (Kritis). Membuka celah Supply Chain Attack berskala masif bagi perusahaan yang akun developer-nya diambil alih. Ini juga berdampak pada potensi kebocoran data proyek |

|                        |                                                                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>Durasi Serangan</b> | Kerentanan ada di dalam sistem selama sekitar 8 bulan (Sejak rilis versi 16.1 pada Mei 2023 hingga patch dirilis pada Januari 2024) |
| <b>Cara Ditemukan</b>  | Dilaporkan oleh security researcher independen (alias: Asterion) melalui program bug bounty GitLab di HackerOne                     |

## 1.2 Kasus yang Dipilih

Kasus *Account Takeover via Password Reset without user interactions* pada GitLab CE/EE. GitLab mengumumkan kasus ini dalam *critical security release* pada 11 Januari 2024. Menurut *advisory* resmi GitLab, kerentanan ini memengaruhi versi 16.1 hingga sebelum versi patch terkait, dan memungkinkan email *reset password* terkirim ke alamat email yang belum terverifikasi, sehingga membuka peluang pengambilalihan akun tanpa interaksi dari pengguna. GitLab juga menjelaskan bahwa kerentanan ini diberi CVE-2023-7028 dan diperkenalkan pada GitLab 16.1.0 tanggal 1 Mei 2023.

Secara teknis, laporan yang dipublikasikan GitLab menunjukkan bahwa penyerang dapat memanipulasi *request* pada fitur *forgot password* dengan memasukkan lebih dari satu alamat email, yaitu email korban dan email penyerang, sehingga sistem mengirimkan tautan *reset password* ke kedua alamat tersebut. Setelah menerima tautan tersebut, penyerang dapat mengganti *password* akun korban dan masuk ke akun tersebut. GitLab menyatakan bahwa akar masalahnya berkaitan dengan bug pada proses verifikasi email setelah perubahan yang dibuat agar pengguna bisa melakukan *reset password* melalui *secondary email address*.

Kerentanan ini menunjukkan bahwa model pemulihan akun masih mengizinkan *state* yang seharusnya tidak valid untuk terjadi di dalam sistem. Secara konkret, sistem tidak memiliki mekanisme yang memastikan bahwa tautan *reset password* hanya dapat dikirimkan ke alamat email yang benar-benar terdaftar dan terverifikasi sebagai milik akun yang bersangkutan. Kondisi ini memperlihatkan bahwa ruang *state* yang valid tidak pernah didefinisikan secara eksplisit dalam desain sistem, sehingga kondisi berbahaya seperti pengiriman token ke email pihak ketiga dapat terjadi tanpa memicu penolakan apapun dari sistem.

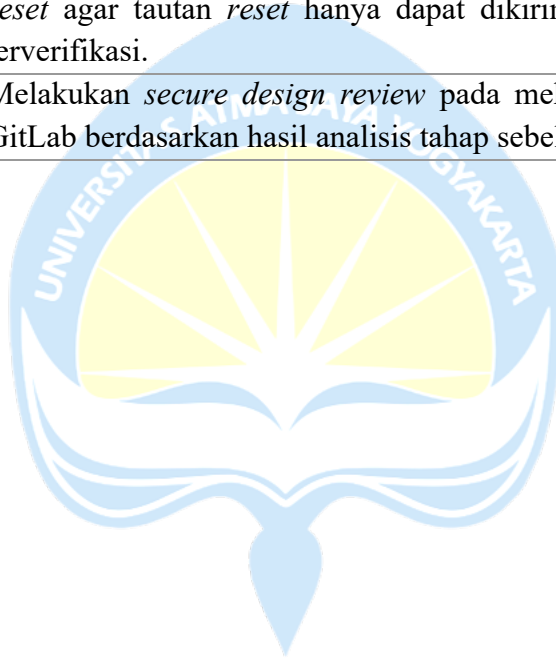
Kasus ini menunjukkan adanya pelanggaran mendasar pada aspek otentikasi, di mana mekanisme pemulihan akun tidak dapat memastikan bahwa hanya entitas yang sah yang berhak melakukan *reset password*. Lebih jauh, sistem juga tidak membedakan antara alamat email yang telah terverifikasi dengan alamat yang belum terverifikasi, sehingga keduanya diperlakukan secara setara dalam proses pengiriman token. Kombinasi kedua kelemahan ini memungkinkan penyerang untuk mengambil alih akun korban sepenuhnya tanpa memerlukan interaksi apapun dari pihak korban, hanya dengan memanfaatkan celah pada lapisan validasi input yang tidak ketat.

### 1.3 Tujuan Tugas

Tugas besar ini disusun dalam tiga tahap yang saling berkaitan, mulai dari analisis akar masalah, perancangan perbaikan berbasis *state machine* dan *invariant*, hingga evaluasi desain keamanan secara menyeluruh. Setiap tahap dirancang untuk membangun pemahaman yang bertahap dan sistematis terhadap kerentanan CVE-2023-7028 pada GitLab. Rincian tujuan dari masing-masing tahap dapat dilihat pada Tabel 1.2 berikut.

Tabel 1.2 Tujuan Laporan

| Tahap   | Tujuan                                                                                                                                                                           |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tahap 1 | Menganalisis <i>root cause</i> kerentanan <i>password reset</i> GitLab, mengidentifikasi state berbahaya, dan merancang perbaikan dengan <i>invariant/domain primitive</i> .     |
| Tahap 2 | Merancang <i>state machine</i> dan <i>temporal invariant</i> pada proses <i>password reset</i> agar tautan <i>reset</i> hanya dapat dikirim ke email yang sah dan terverifikasi. |
| Tahap 3 | Melakukan <i>secure design review</i> pada mekanisme <i>account recovery</i> GitLab berdasarkan hasil analisis tahap sebelumnya.                                                 |



## 2 Root Cause Reconstruction & Invariant Design

Analisis pada bagian ini berfokus pada rekonstruksi akar penyebab kerentanan CVE-2023-7028 berdasarkan perilaku sistem saat memproses permintaan *password reset*, serta perancangan ulang model domain menggunakan *invariant* untuk mencegah terulangnya kondisi berbahaya yang sama. Rekonstruksi dilakukan dengan menyusun ulang alur kode dan *state* yang terlibat, sehingga hubungan antara keputusan desain awal dan permukaan serangan dapat diidentifikasi secara eksplisit. Berdasarkan hasil rekonstruksi tersebut, disusun seperangkat *invariant* dan *domain primitive* yang memaksa sistem hanya beroperasi dalam *valid state space*, sehingga *state* yang tidak valid secara domain tidak lagi dapat terbentuk meskipun terjadi kesalahan pada lapisan implementasi.

### 2.1 Model Domain Awal (Sebelum Perbaikan)

Berdasarkan analisis yang dipublikasikan oleh tim GitLab, kerentanan ini berakar pada cara parameter email diproses pada *endpoint password reset*. Secara normal, sistem mengharapkan parameter `user[email]` berupa sebuah `string` tunggal berisi satu alamat email terverifikasi. Namun karena tidak ada *validasi* yang memastikan bahwa nilai tersebut adalah `string` (bukan `list/array`), penyerang dapat mengirimkan dua alamat email sekaligus dalam satu permintaan, dan sistem akan memproses keduanya serta mengirimkan *token reset* ke seluruh email dalam daftar tersebut.

Untuk memahami permasalahan ini, rekonstruksi kode berikut disusun dalam konteks *Java Spring Boot* sebagai representasi modern dari arsitektur MVC (*Model View Controller*) yang digunakan GitLab. MVC adalah pola arsitektur perangkat lunak yang memisahkan aplikasi menjadi tiga komponen utama: *Model* yang bertanggung jawab mengelola data dan logika bisnis, *View* yang menangani tampilan antarmuka pengguna, dan *Controller* yang berperan sebagai perantara antara *Model* dan *View* dengan memproses setiap *request* yang masuk dari pengguna. Berikut adalah rekonstruksi kode yang merepresentasikan pola desain bermasalah tersebut.

```
@RestController
@RequestMapping("/users")
public class PasswordResetController {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordResetService passwordResetService;

    @Autowired
    private EmailService emailService;

    @PostMapping("/password")
    public ResponseEntity<String> requestPasswordReset(
        @RequestBody PasswordResetRequest request) {

        Object emailParam = request.getEmail();
```

```

    List<String> emailTargets = resolveEmailTargets(emailParam);

    for (String email : emailTargets) {
        User user = userRepository.findByPrimaryEmail(emailTargets.get(0));

        if (user != null) {
            String resetToken = passwordResetService.generateToken(user);

            emailService.sendPasswordResetEmail(email, resetToken);
        }
    }

    return ResponseEntity.ok("Instruksi reset kata sandi telah dikirim.");
}

private List<String> resolveEmailTargets(Object emailParam) {
    if (emailParam instanceof List) {
        return (List<String>) emailParam;
    }
    return Collections.singletonList((String) emailParam);
}
}

public class PasswordResetRequest {
    private Object email;

    public Object getEmail() { return email; }
    public void setEmail(Object email) { this.email = email; }
}

@Service
public class PasswordResetService {

    @Autowired
    private UserRepository userRepository;

    public String generateToken(User user) {
        String token = UUID.randomUUID().toString();

        user.setResetPasswordToken(token);
        user.setResetPasswordSentAt(LocalDate.now());
        userRepository.save(user);

        return token;
    }

    public boolean resetPassword(String token, String newPassword) {
        User user = userRepository.findByResetPasswordToken(token);

        if (user == null) return false;
    }
}

```

```

        if
        (user.getResetPasswordSentAt().plusHours(2).isBefore(LocalDateTime.now())) {
            return false;
        }

        user.setPassword(passwordEncoder.encode(newPassword));
        user.setResetPasswordToken(null);
        user.setResetPasswordSentAt(null);
        userRepository.save(user);
        return true;
    }
}

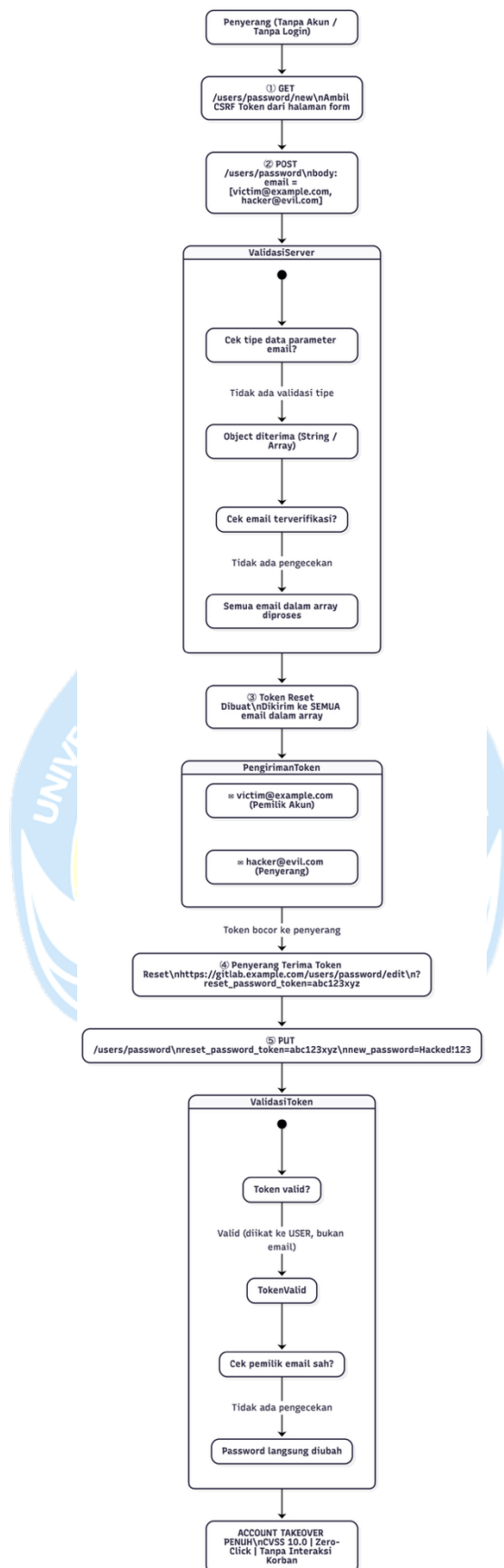
```

Berdasarkan ketiga kelas di atas, terdapat rantai kesalahan yang saling memperparah. Pertama, model *request* menggunakan Object sebagai tipe data email, sehingga *JSON deserializer* menerima baik String maupun List tanpa penolakan. Kedua, *controller* memproses semua alamat dalam *list* tersebut dan mengirimkan token yang sama ke semuanya termasuk alamat email penyerang. Ketiga, *service* tidak mengikat token ke penerima yang sah, sehingga token yang diterima penyerang dapat langsung digunakan untuk mereset kata sandi akun korban.

### 2.1.1 State Diagram Berbahaya

Untuk memperjelas bagaimana celah kerentanan CVE-2023-7028 dapat dieksploitasi secara teknis, alur serangan direpresentasikan dalam bentuk diagram *state* berbahaya. Diagram ini menggambarkan seluruh tahapan yang dilalui penyerang, mulai dari pengambilan token *CSRF* melalui *endpoint* GET /users/password/new, pengiriman *payload* array berisi dua alamat email sekaligus, hingga sistem memproses kedua alamat tersebut tanpa validasi tipe data maupun verifikasi kepemilikan email.

Akibat tidak adanya pengecekan pada lapisan validasi *server*, token reset yang sah dikirimkan ke seluruh alamat dalam array, termasuk alamat milik penyerang. Penyerang kemudian menggunakan token tersebut untuk mengakses *endpoint* PUT /users/password dan mengganti kata sandi akun korban tanpa sepengetahuannya. Kondisi ini mengakibatkan *account takeover* penuh dengan skor CVSS 10.0 tanpa memerlukan interaksi apapun dari korban. Alur lengkap serangan ini dapat dilihat pada Gambar 2.1 berikut.



Gambar 2.1 Diagram State Berbahaya

### 2.1.2 State Berbahaya yang Dapat Direpresentasikan

Berdasarkan rekonstruksi kode yang telah dijabarkan sebelumnya, terdapat beberapa kondisi berbahaya yang secara struktural dapat direpresentasikan oleh sistem tanpa memicu *error* atau penolakan apapun. Kondisi-kondisi ini muncul bukan karena kesalahan logika tunggal, melainkan karena tidak adanya aturan domain yang membatasi ruang *state* yang valid pada alur *password reset*. Dalam prinsip *secure by design*, setiap kondisi yang seharusnya tidak boleh terjadi harus dibuat tidak mungkin direpresentasikan secara struktural, bukan hanya dicegah melalui pemeriksaan *runtime*. Daftar *state* berbahaya yang berhasil diidentifikasi dari sistem yang rentan dapat dilihat pada Tabel 2.1 berikut.

Tabel 2.1 State yang Dapat Direpresentasikan

| State Berbahaya                                                           | Deskripsi                                                                                                                                                                                            |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>emails</code> bertipe <code>Object</code> bukan <code>String</code> | Sistem menerima array sebagai nilai field email tanpa validasi tipe data, sehingga payload <code>{"email":["victim@...", "hacker@evil.com"]}</code> diproses secara normal oleh Jackson deserializer |
| <code>emailTargets.size() &gt; 1</code>                                   | Satu request reset password dapat memiliki lebih dari satu tujuan pengiriman token. Tidak ada aturan domain yang membatasi ini harus tepat satu alamat                                               |
| <code>emailTargets.get(i) != user.getPrimaryEmail()</code>                | Alamat email dalam daftar pengiriman tidak harus sama dengan email utama yang terdaftar pada akun pengguna yang bersangkutan                                                                         |
| <code>reset_token</code> valid untuk semua penerima email                 | Token yang dihasilkan tidak terikat secara eksklusif kepada email utama pengguna; token tersebut dapat digunakan oleh siapapun yang menerimanya                                                      |
| <code>PasswordResetRequest</code> dibuat dari input eksternal             | Objek <code>PasswordResetRequest</code> membangun daftar targetnya dari input request HTTP secara langsung, bukan dari data tepercaya di dalam Aggregate Root ( <code>User</code> )                  |

## 2.2 Analisis Root Cause

Isu utama dalam kerentanan ini adalah tidak adanya validasi tipe data pada parameter input yang digunakan dalam alur kritis autentikasi. Sistem secara implisit mempercayai bahwa parameter email selalu berupa string tunggal, tanpa pernah memvalidasi asumsi tersebut secara eksplisit di lapisan manapun. Ketiadaan validasi ini membuka celah bagi penyerang untuk menyuntikkan nilai bertipe array yang kemudian diproses oleh sistem seolah-olah merupakan input yang sah.

Kerentanan CVE-2023-7028 tidak muncul dari satu kesalahan tunggal, melainkan dari akumulasi tiga kelemahan desain yang saling memperparah satu sama lain. Ketiga kelemahan ini berada pada lapisan yang berbeda, yaitu lapisan model *request*, lapisan *controller*, dan lapisan *service*, sehingga bahkan jika salah satu lapisan diperbaiki secara parsial, lapisan lainnya tetap dapat dieksploitasi oleh penyerang. Berikut adalah uraian masing-masing kelemahan desain yang menjadi akar penyebab kerentanan ini.

### 1. Parameter Email Tidak Divalidasi Tipe Datanya

Model request menggunakan Object sebagai tipe field email, sehingga Jackson (JSON deserializer Spring Boot) menerima baik String maupun List<String> dari payload tanpa melempar exception. Validasi tipe data sama sekali tidak ada.

```
private Object email;
```

### 2. Iterasi Daftar Email Tanpa Verifikasi Kepemilikan

Controller mengiterasi seluruh alamat email dalam daftar dan mengirimkan token yang sama ke semuanya, termasuk alamat yang tidak terdaftar dan belum diverifikasi sebagai milik akun mana pun.

```
for (String email : emailTargets) {  
    emailService.sendPasswordResetEmail(email, resetToken);  
}
```

### 3. Token Reset Tidak Terikat pada Penerima yang Sah

Token reset hanya disimpan bersama data akun pengguna, tanpa menyimpan informasi untuk email mana token tersebut dimaksudkan. Akibatnya, siapa pun yang memegang token, baik pemilik akun maupun penyerang dapat menggunakannya untuk mengubah kata sandi.

```
user.setResetPasswordToken(token);  
user.setResetPasswordSentAt(LocalDateTime.now());  
User user = userRepository.findByResetPasswordToken(stolenToken);
```

### 2.2.1 Hubungan ke Konsep Valid State Space

Dalam prinsip *secure by design*, setiap sistem yang aman harus dirancang sedemikian rupa sehingga kondisi yang tidak valid secara domain tidak dapat direpresentasikan sama sekali, bukan hanya dicegah melalui pemeriksaan logika *runtime*. Konsep *valid state space* mengacu pada himpunan seluruh kondisi yang secara logika bisnis diizinkan untuk ada di dalam sistem. Apabila sistem mengizinkan kondisi di luar himpunan tersebut untuk terjadi, maka sistem tersebut memiliki celah struktural yang dapat dieksploitasi meskipun tidak ada *bug* logika yang tampak secara eksplisit.

Pada kasus CVE-2023-7028, terdapat dua kondisi yang menggambarkan perbedaan antara *state* berbahaya yang dapat direpresentasikan dengan *valid state space* yang seharusnya ditinggalkan oleh sistem, yaitu sebagai berikut.

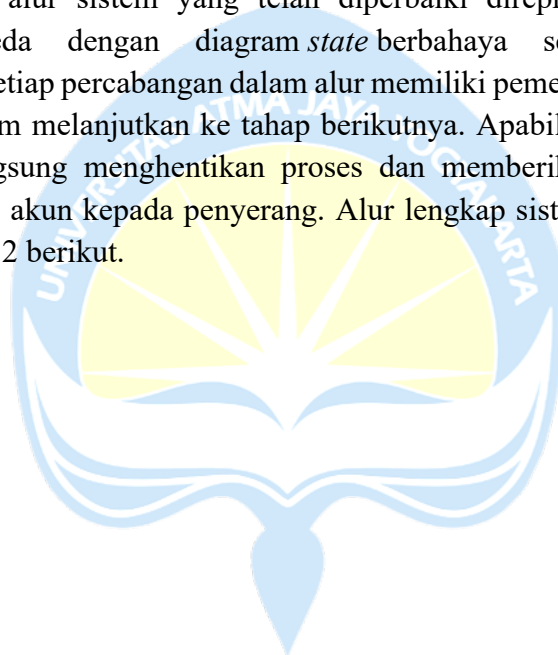
- **State berbahaya yang bisa direpresentasikan:**  
`PasswordResetToken(delivered_to="attacker@email.com",  
account_owner="victim@email.com")` dianggap valid oleh sistem meskipun secara logika bisnis kondisi ini sangat berbahaya, karena token milik akun korban dikirimkan ke alamat email yang bukan miliknya.
- **Valid state space yang benar:** Token reset hanya boleh ada jika `delivery_email` merupakan bagian dari `verified_emails` milik identitas pengguna yang bersangkutan, sehingga secara struktural tidak mungkin token dikirimkan ke alamat yang tidak terverifikasi sebagai milik akun tersebut.

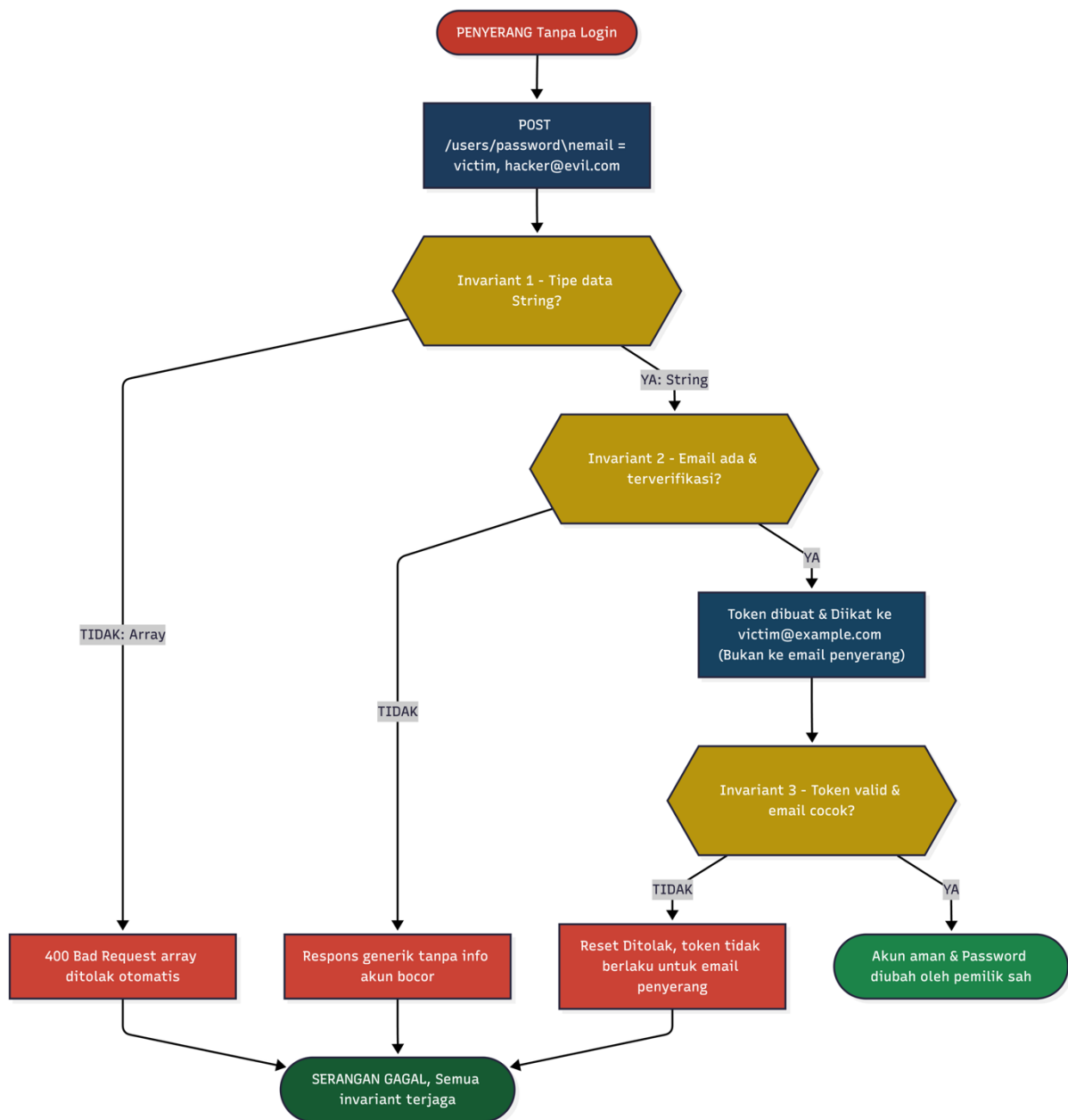
## 2.3 Perbaikan dengan Invariant/Domain Primitive

Setelah mengidentifikasi akar penyebab kerentanan CVE-2023-7028, langkah selanjutnya adalah merancang perbaikan yang tidak hanya menambal celah pada satu titik, tetapi menutup seluruh ruang *state* berbahaya secara struktural. Pendekatan yang digunakan mengacu pada prinsip *make invalid state unrepresentable*, di mana setiap kondisi yang berbahaya dibuat tidak mungkin terjadi sejak objek dibentuk, bukan hanya dicegah melalui pemeriksaan *runtime* yang dapat dilewati. Perbaikan ini diterapkan melalui empat *invariant* utama yang bekerja secara berlapis, mulai dari validasi tipe data pada lapisan *request*, verifikasi kepemilikan email, pengikatan token ke penerima yang sah, hingga pembatasan laju permintaan sebagai lapisan pertahanan tambahan.

### 2.3.1 Diagram State Setelah Perbaikan

Untuk memvisualisasikan bagaimana keempat *invariant* bekerja secara terpadu dalam menangkal serangan, alur sistem yang telah diperbaiki direpresentasikan dalam bentuk diagram *state*. Berbeda dengan diagram *state* berbahaya sebelumnya, diagram ini menunjukkan bahwa setiap percabangan dalam alur memiliki pemeriksaan eksplisit yang harus dilewati sebelum sistem melanjutkan ke tahap berikutnya. Apabila salah satu *invariant* tidak terpenuhi, sistem langsung menghentikan proses dan memberikan respons generik tanpa mengekspos informasi akun kepada penyerang. Alur lengkap sistem setelah perbaikan dapat dilihat pada Gambar 2.2 berikut.





Gambar 2.2 Diagram State Perbaikan

### 2.3.2 Aturan Domain

Perbaikan terhadap kerentanan CVE-2023-7028 tidak cukup dilakukan hanya dengan menambahkan pemeriksaan kondisional pada lapisan *controller* atau *service*. Pendekatan yang lebih kokoh adalah dengan menetapkan aturan domain yang bersifat *invariant*, yaitu kondisi yang harus selalu benar sepanjang siklus hidup objek dan tidak dapat dilanggar melalui cara apapun, termasuk dari *input* eksternal. Aturan-aturan ini ditegakkan langsung di dalam konstruktor objek sehingga instansiasi objek yang tidak valid secara domain tidak mungkin berhasil dilakukan.

Aturan domain yang dirancang dalam perbaikan ini dikelompokkan ke dalam empat kategori, yaitu Format, Temporal, Ownership, dan Kardinalitas. Kategori Format memastikan bahwa tipe dan struktur data yang masuk ke sistem selalu valid secara teknis. Kategori Temporal mengatur batasan waktu dan penggunaan token agar tidak dapat disalahgunakan di luar jendela waktu yang sah. Kategori *Ownership* memastikan bahwa token hanya dapat dikirimkan ke alamat email yang benar-benar terverifikasi sebagai milik akun yang bersangkutan. Kategori Kardinalitas memastikan bahwa setiap permintaan reset hanya memiliki tepat satu alamat tujuan pengiriman. Rincian seluruh aturan domain beserta *bug* yang dicegah dapat dilihat pada Tabel 2.2 berikut.

Tabel 2.2 Aturan Domain

| Kategori         | Invariant                                                                                                                                                                                       | Bug yang Dicegah                                                                                        |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Format</b>    | VerifiedPrimaryEmail harus berupa string tunggal (bukan List atau tipe lainnya) dengan format email valid sesuai RFC 5321; divalidasi ketat di konstruktor menggunakan regex                    | Mencegah type confusion dan array injection inti dari CVE-2023-7028                                     |
| <b>Format</b>    | reset_token harus dihasilkan menggunakan Cryptographically Secure Pseudo-Random Number Generator (CSPRNG) dengan panjang minimum 32 byte                                                        | Mencegah token guessing dan serangan brute-force terhadap token reset                                   |
| <b>Temporal</b>  | PasswordResetRequest wajib memiliki expires_at yang ditetapkan pada saat pembuatan; masa berlaku maksimum adalah 15 menit; objek tidak dapat dibuat tanpa nilai ini                             | Mencegah penggunaan token reset yang tertunda atau dicuri untuk digunakan di kemudian waktu             |
| <b>Temporal</b>  | Satu token reset hanya dapat digunakan satu kali (single-use); setelah digunakan, status token berubah menjadi CONSUMED dan tidak dapat digunakan lagi                                          | Mencegah serangan replay di mana token yang sudah digunakan dieksploitasi kembali                       |
| <b>Ownership</b> | VerifiedPrimaryEmail hanya dapat diinstansiasi melalui method User.get_verified_primary_email() tidak dapat dibuat secara langsung dari input request eksternal (HTTP body, query string, dll.) | Mencegah penyerang menyuntikkan email arbitrer sebagai tujuan pengiriman token, akar dari CVE-2023-7028 |

|                    |                                                                                                                                                                   |                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ownership</b>   | PasswordResetRequest hanya menerima VerifiedPrimaryEmail sebagai parameter konstruktor, bukan <code>str</code> atau <code>List[str]</code>                        | Menjamin secara struktural bahwa tujuan pengiriman token selalu merupakan email yang telah terverifikasi sebagai milik pengguna yang bersangkutan |
| <b>Kardinalita</b> | Setiap PasswordResetRequest memiliki tepat satu <code>_destination</code> bertipe VerifiedPrimaryEmail tidak ada field untuk menyimpan beberapa tujuan pengiriman | Memastikan token tidak pernah dapat dikirimkan ke lebih dari satu alamat secara bersamaan, menutup vektor serangan CVE-2023-7028                  |

## 2.4 Perancangan Perbaikan dengan Invariant

Pendekatan yang diterapkan dalam perbaikan ini berlandaskan pada prinsip *make invalid state unrepresentable*, yaitu sistem dirancang agar kondisi berbahaya tidak mungkin terjadi secara struktural, bukan hanya dicegah melalui pemeriksaan logika *runtime* yang berpotensi dilewati. Dengan demikian, pelanggaran aturan domain akan langsung menghasilkan kegagalan pada saat instansiasi objek, sebelum logika bisnis apapun sempat dieksekusi.

Perbaikan diimplementasikan melalui empat *invariant* yang bekerja secara berlapis dan saling melengkapi, mulai dari validasi tipe data, verifikasi kepemilikan email, pengikatan token ke penerima yang sah, hingga pembatasan laju permintaan sebagai pertahanan tambahan. Berikut adalah rincian implementasi masing-masing *invariant*.

### 2.4.1 Invariant 1: Validasi Tipe Data Parameter Email Secara Eksplisit

Field email pada model request harus bertipe `String`, bukan `Object`, sehingga Jackson secara struktural langsung menolak payload yang berisi array tanpa memerlukan logika validasi tambahan.

```
public class PasswordResetRequest {
    @NotBlank(message = "Email tidak boleh kosong")
    @Email(message = "Format email tidak valid")
    @JsonDeserialize(using = StrictStringDeserializer.class)
    private String email;
```

```

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}

public class StrictStringDeserializer extends JsonSerializer<String> {

    @Override
    public String deserialize(JsonParser p, DeserializationContext ctx)
        throws IOException {

        if (p.getCurrentToken() != JsonToken.VALUE_STRING) {
            throw new JsonParseException(p, "p.getCurrentToken().name()");
        }
        return p.getValueAsString();
    }
}

```

#### 2.4.2 Invariant 2: Email Tujuan Harus Terverifikasi Milik Akun

Sistem harus memastikan bahwa token reset **hanya** dikirimkan ke alamat email yang telah terdaftar dan terverifikasi pada akun yang bersangkutan. Alamat email sembarang dari luar akun tidak boleh menerima token apa pun.

```

@PostMapping("/password")
public ResponseEntity<String> requestPasswordReset(
    @Valid @RequestBody PasswordResetRequest request) {

    String emailInput = request.getEmail().trim().toLowerCase();

    Optional<User> optionalUser = userRepository.findByVerifiedEmail(emailInput);

    if (optionalUser.isEmpty()) {
        return ResponseEntity.ok("Jika email terdaftar, instruksi telah dikirim.");
    }

    User user = optionalUser.get();

    String verifiedDestination = user.getPrimaryVerifiedEmail();

    String token = passwordResetService.generateBoundToken(user,
        verifiedDestination);
    emailService.sendPasswordResetEmail(verifiedDestination, token);

    return ResponseEntity.ok("Jika email terdaftar, instruksi telah dikirim.");
}

```

### 2.4.3 Invariant 3: Token Reset Terikat pada Penerima Email yang Spesifik

Token reset harus disimpan bersama informasi email sah yang menjadi penerima, sehingga token yang bocor ke pihak tidak berwenang tidak dapat digunakan untuk mereset kata sandi akun target.

```
@PostMapping("/password")
public ResponseEntity<String> requestPasswordReset(
    @Valid @RequestBody PasswordResetRequest request) {

    String emailInput = request.getEmail().trim().toLowerCase();

    Optional<User> optionalUser = userRepository.findByVerifiedEmail(emailInput);

    if (optionalUser.isEmpty()) {
        return ResponseEntity.ok("Jika email terdaftar, instruksi telah dikirim.");
    }

    User user = optionalUser.get();

    String verifiedDestination = user.getPrimaryVerifiedEmail();

    String token = passwordResetService.generateBoundToken(user,
verifiedDestination);
    emailService.sendPasswordResetEmail(verifiedDestination, token);

    return ResponseEntity.ok("Jika email terdaftar, instruksi telah dikirim.");
}
```

### 2.4.4 Invariant 4: Rate Limiting dan Monitoring

Sebagai lapisan pertahanan tambahan, sistem harus membatasi jumlah permintaan reset kata sandi per alamat email per satuan waktu, serta mencatat dan memantau request yang mencurigakan, khususnya payload yang mengandung array email.

```
@Component
public class PasswordResetRateLimiter {

    private final Cache<String, Integer> attemptCache = CacheBuilder.newBuilder()
        .expireAfterWrite(1, TimeUnit.HOURS)
        .build();

    private static final int MAX_ATTEMPTS_PER_HOUR = 5;

    public boolean isRateLimited(String email) {
        Integer attempts = attemptCache.getIfPresent(email.toLowerCase());
        return attempts != null && attempts >= MAX_ATTEMPTS_PER_HOUR;
    }
}
```

```

    }

    public void recordAttempt(String email) {
        String key = email.toLowerCase();
        Integer current = attemptCache.getIfPresent(key);
        attemptCache.put(key, current == null ? 1 : current + 1);
    }
}

@PostMapping("/password")
public ResponseEntity<String> requestPasswordReset(
    @Valid @RequestBody PasswordResetRequest request,
    HttpServletRequest httpRequest) {

    String email = request.getEmail().trim().toLowerCase();

    if (rateLimiter.isRateLimited(email)) {
        return ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS)
            .body("Terlalu banyak percobaan. Coba lagi dalam 1 jam.");
    }

    rateLimiter.recordAttempt(email);

    log.info("[PASSWORD_RESET] email={} ip={} userAgent={}",
        email,
        httpRequest.getRemoteAddr(),
        httpRequest.getHeader("User-Agent"));

    return processPasswordReset(email);
}

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<String> handleMalformedRequest(
        HttpMessageNotReadableException ex,
        HttpServletRequest request) {

        log.warn("[SECURITY_ALERT] Payload tidak valid pada {}: {} dari IP {}",
            request.getRequestURI(),
            ex.getMessage(),
            request.getRemoteAddr());

        return ResponseEntity.badRequest()
            .body("Format request tidak valid.");
    }
}

```

### 3 State Machine & Temporal Invariant Project

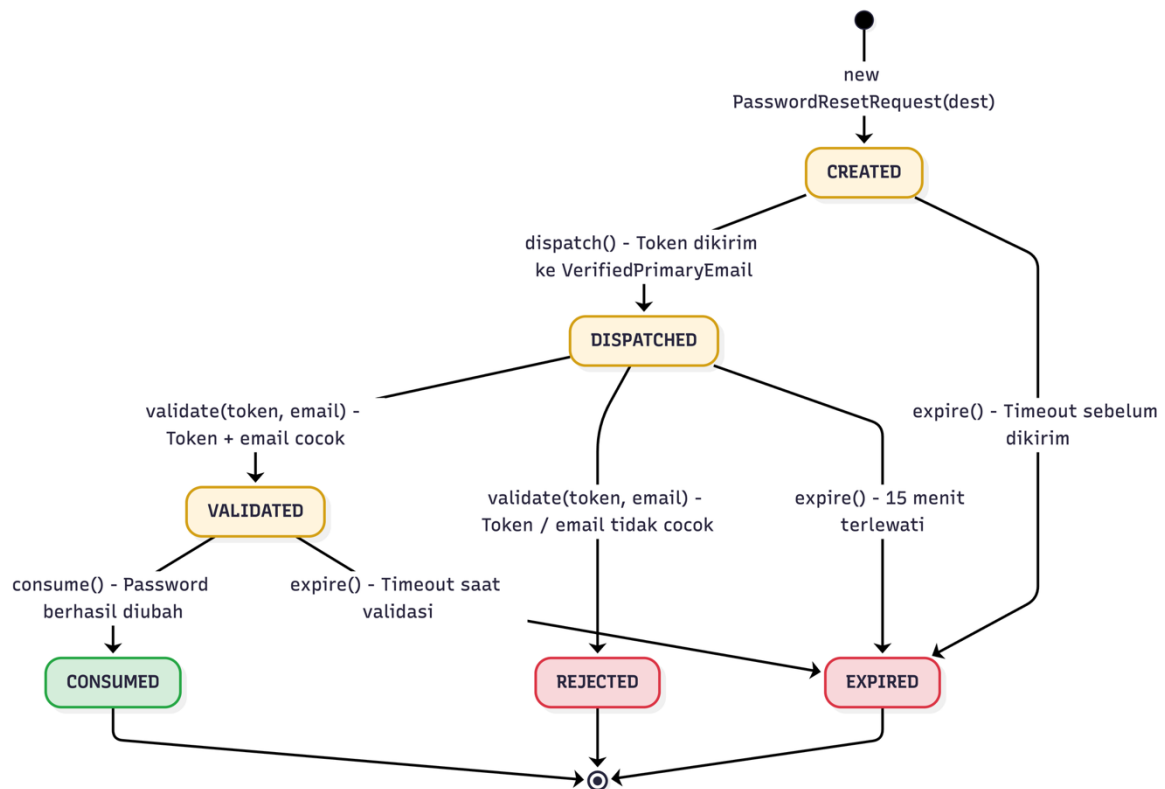
Kerentanan CVE-2023-7028 pada GitLab bukan sekadar persoalan validasi input yang terlewat, melainkan cerminan dari tidak adanya model yang mendefinisikan secara tegas bagaimana sebuah permintaan reset password seharusnya berperilaku sepanjang hidupnya. Sistem yang rentan memperlakukan proses reset sebagai serangkaian pemanggilan fungsi yang berdiri sendiri, tanpa ada kontrak yang mengatur urutan mana yang harus dipatuhi. Akibatnya, token dapat dikirim ke lebih dari satu alamat email sekaligus, dan sistem tidak memiliki mekanisme untuk menolak penggunaan token tersebut meskipun ia berada di tangan orang yang salah.

Pendekatan yang dipilih dalam proyek ini adalah memodelkan entitas PasswordResetRequest sebagai sebuah state machine. Entitas ini dipilih karena memiliki siklus hidup yang jelas dan berurutan: dibuat, dikirim, divalidasi, lalu dikonsumsi atau kedaluwarsa. Setiap tahapan memiliki prasyarat yang harus dipenuhi sebelum tahapan berikutnya dapat dimulai. Dengan menegakkan urutan ini sebagai *temporal invariant* yang tertanam langsung di dalam domain object, sistem tidak lagi bergantung pada asumsi bahwa semua pemanggil akan mengikuti urutan yang benar, melainkan secara aktif menolak setiap eksekusi yang melanggar kontrak tersebut.

#### 3.1 Diagram State dan Transisi State

Untuk memastikan bahwa seluruh *invariant* yang telah dirancang benar-benar ditegakkan sepanjang siklus hidup permintaan *password reset*, entitas PasswordResetRequest dimodelkan sebagai sebuah *state machine* eksplisit. Setiap perubahan *state* hanya dapat terjadi melalui transisi yang telah ditentukan, sehingga urutan operasi seperti pembuatan token, pengiriman email, validasi, dan konsumsi token tidak dapat dilompati atau dijalankan di luar konteks yang semestinya.

Diagram *state* dan transisi pada bagian ini menggambarkan bagaimana objek PasswordResetRequest berpindah dari *state* CREATED ke DISPATCHED, lalu ke VALIDATED, dan akhirnya berakhir pada *state* terminal CONSUMED, REJECTED, atau EXPIRED. Setiap transisi dipicu oleh operasi spesifik seperti `dispatch()`, `validate()`, atau `consume()`, dan hanya diizinkan apabila *state* saat ini memenuhi aturan yang telah didefinisikan sebelumnya. Alur lengkap hubungan antar *state* dan transisinya dapat dilihat pada Gambar 3.1 berikut.



Gambar 3.1 Diagram State dan Transisi State

## 3.2 Enum State

Representasi state yang tersebar sebagai konstanta string atau integer rentan terhadap kesalahan assignment dan tidak memberikan jaminan compile-time. Oleh karena itu, seluruh state dikonsolidasikan ke dalam satu enum terpusat yang juga menyimpan logika transisi yang diizinkan. Pendekatan ini menjadikan pelanggaran urutan state terdeteksi sebagai `IllegalStateException` pada saat runtime, bukan sebagai bug yang baru tampak setelah serangan terjadi.

```

public enum PasswordResetState {

    CREATED,
    DISPATCHED,
    VALIDATED,
    CONSUMED,
    EXPIRED,
    REJECTED;

    public boolean canTransitionTo(PasswordResetState next) {
        return switch (this) {
            case CREATED    -> next == DISPATCHED || next == EXPIRED;
            case DISPATCHED -> next == VALIDATED  || next == EXPIRED
                               || next == REJECTED;
            case VALIDATED  -> next == CONSUMED    || next == EXPIRED;
            case CONSUMED, EXPIRED, REJECTED -> false;
        };
    }
}

```

```

    }

    public boolean isTerminal() {
        return this == CONSUMED || this == EXPIRED || this == REJECTED;
    }
}

```

Dengan menggunakan enum ini, penambahan state baru di masa mendatang mewajibkan developer untuk secara eksplisit mendefinisikan transisi yang diizinkan. Sistem tidak akan mengkompilasi dengan benar apabila ada cabang switch yang terlewat, sehingga temporal invariant terjaga secara struktural, bukan bergantung pada kedisiplinan developer semata.

### 3.3 Temporal Invariant

Temporal *invariant* merupakan aturan yang menegaskan bahwa suatu operasi hanya boleh dieksekusi jika sistem sedang berada dalam *state* yang tepat dan dalam urutan waktu yang benar. Pada kasus CVE-2023-7028, tidak adanya *temporal invariant* membuat GitLab memproses token reset tanpa peduli apakah email tujuan sudah diverifikasi sebagai milik akun tersebut, dan tanpa memastikan bahwa urutan *state* telah dilalui secara benar. Dengan kata lain, sistem tidak memiliki “kontrak waktu” yang mengatur kapan sebuah token boleh dibuat, dikirim, divalidasi, dan dikonsumsi.

Untuk menutup celah ini, proses *password reset* harus dimodelkan sebagai rangkaian *state* yang hanya dapat dilalui melalui transisi yang sah, misalnya dari `CREATED` ke `DISPATCHED`, kemudian ke `VALIDATED`, dan berakhir di `CONSUMED` atau `EXPIRED`. Setiap pemanggilan metode seperti `dispatch()`, `validate()`, atau `consume()` wajib memeriksa *state* saat ini dan menolak eksekusi apabila urutannya tidak sesuai dengan aturan yang telah didefinisikan. Berikut adalah implementasi bertahap yang menerapkan *temporal invariant* pada setiap titik kritis dalam alur *password reset*.

#### 3.3.1 Definisi Struktur Class Utama

Kerentanan CVE-2023-7028 bermula dari fakta bahwa GitLab menerima field `user[email]` sebagai tipe `Object`, sehingga array seperti `["korban@example.com", "hacker@evil.com"]` diperlakukan layaknya input biasa. Untuk menutup celah ini secara struktural, dibuat value object `VerifiedPrimaryEmail` yang hanya dapat dikonstruksi melalui `User` aggregate root, bukan dari raw HTTP input. Seluruh domain object kemudian dibangun di atas fondasi ini.

```

public final class VerifiedPrimaryEmail {

    private static final String EMAIL_REGEX =
        "^[a-zA-Z0-9._%+\\-]+@[a-zA-Z0-9\\.\\-]+\\. [a-zA-Z]{2,}$";
}

```

```

private final String value;

VerifiedPrimaryEmail(String rawEmail) {
    if (rawEmail == null || !rawEmail.matches(EMAIL_REGEX)) {
        throw new IllegalArgumentException("Format email tidak valid");
    }
    this.value = rawEmail.trim().toLowerCase();
}

public String getValue() { return value; }
}

public final class PasswordResetRequest {

    private final VerifiedPrimaryEmail destination;
    private final String tokenHash;
    private final LocalDateTime expiresAt;
    private PasswordResetState state;

    private void transitionTo(PasswordResetState next) {
        if (!this.state.canTransitionTo(next)) {
            throw new IllegalStateException(
                "Transisi tidak diizinkan: " + this.state + " -> " + next);
        }
        this.state = next;
    }
}

```

### 3.3.2 Tambahkan Initial State di Konstruktor

Setiap instance `PasswordResetRequest` harus lahir dalam state `CREATED`, tidak ada cara lain untuk membuat objek ini selain melalui konstruktor ini. Token dihasilkan menggunakan `SecureRandom` dengan entropi minimum 32 byte untuk mencegah serangan brute-force, dan batas waktu 15 menit ditetapkan langsung di titik konstruksi sehingga token tidak pernah ada dalam kondisi tanpa expiry.

```

public PasswordResetRequest(VerifiedPrimaryEmail destination) {
    Objects.requireNonNull(destination, "Destination tidak boleh null");
    this.destination = destination;

    byte[] tokenBytes = new byte[32];
    new SecureRandom().nextBytes(tokenBytes);
    this.tokenHash = Base64.getUrlEncoder()
        .withoutPadding()
        .encodeToString(tokenBytes);

    this.expiresAt = LocalDateTime.now().plusMinutes(15);
}

```

```
        this.state = PasswordResetState.CREATED;
    }
```

### 3.3.3 Tambahkan Pengecekan State di dispatch()

Metode `dispatch()` adalah gerbang pertama dalam alur reset. Ia hanya dapat dieksekusi satu kali, karena setelah state bergerak ke `DISPATCHED`, transisi kembali ke `CREATED` tidak lagi diizinkan oleh `canTransitionTo()`. Hal ini secara langsung mencegah skenario di mana token dikirim ulang ke alamat berbeda setelah pengiriman pertama.

```
public void dispatch() {
    if (LocalDateTime.now().isAfter(this.expiresAt)) {
        transitionTo>PasswordResetState.EXPIRED);
        throw new TokenExpiredException(
            "Token sudah kedaluwarsa sebelum dapat dikirim");
    }

    transitionTo>PasswordResetState.DISPATCHED);
}
```

### 3.3.4 Tambahkan Pengecekan State di validate()

Inilah titik kritis yang tidak ada pada implementasi GitLab yang rentan. Selain memverifikasi bahwa token cocok, `validate()` juga memastikan bahwa email yang diklaim oleh pengguna benar-benar sama dengan destination yang tersimpan di dalam domain object, bukan dari parameter request. Dengan demikian, meskipun penyerang berhasil mendapatkan token, mereka tidak dapat menggunakannya kecuali mereka mengklaim email yang benar, dan email yang benar hanya bisa diklaim oleh pemilik akun.

```
public void validate(String rawToken, String claimedEmail) {
    if (this.state != PasswordResetState.DISPATCHED) {
        throw new IllegalStateException(
            "validate() dipanggil dari state yang tidak valid: " + this.state);
    }

    if (LocalDateTime.now().isAfter(this.expiresAt)) {
        transitionTo>PasswordResetState.EXPIRED);
        throw new TokenExpiredException("Token sudah kedaluwarsa");
    }

    boolean tokenMatch = MessageDigest.isEqual(
        rawToken.getBytes(StandardCharsets.UTF_8),
        this.tokenHash.getBytes(StandardCharsets.UTF_8)
    );
    boolean emailMatch = this.destination.getValue()
```

```

        .equals(claimedEmail.trim().toLowerCase());

    if (!tokenMatch || !emailMatch) {
        transitionTo(PasswordResetState.REJECTED);
        throw new InvalidTokenException("Verifikasi token gagal");
    }

    transitionTo(PasswordResetState.VALIDATED);
}

```

### 3.3.5 Tambahkan Pengecekan State di consume()

`consume()` merupakan operasi final yang mengubah password dan menutup siklus hidup request secara permanen. Pengecekan state di sini berfungsi sebagai lapisan perlindungan terakhir: seandainya ada race condition atau bug di layer atas yang memanggil `consume()` sebelum `validate()` selesai, sistem tetap akan menolak eksekusi karena state-nya tidak berada di `VALIDATED`.

```

public void consume() {
    if (this.state != PasswordResetState.VALIDATED) {
        throw new IllegalStateException(
            "consume() hanya dapat dipanggil setelah validasi berhasil. "
            + "State saat ini: " + this.state);
    }

    if (LocalDateTime.now().isAfter(this.expiresAt)) {
        transitionTo(PasswordResetState.EXPIRED);
        throw new TokenExpiredException(
            "Token kedaluwarsa tepat sebelum dikonsumsi");
    }

    transitionTo(PasswordResetState.CONSUMED);
}

```

### 3.3.6 Tambahkan Pengecekan State Expired

Berbeda dengan `REJECTED` yang dipicu oleh input yang salah, state `EXPIRED` dipicu oleh faktor waktu. `expire()` dirancang idempotent: jika dipanggil berulang kali pada objek yang sudah terminal, ia tidak melakukan apa pun. Sebuah scheduled task memanggil metode ini secara berkala untuk memastikan token basi tidak bertahan di database sebagai permukaan serangan yang tidak perlu.

```

public void expire() {
    if (this.state.isTerminal()) return;

    if (LocalDateTime.now().isAfter(this.expiresAt)) {
        transitionTo>PasswordResetState.EXPIRED);
    }
}

@Component
public class PasswordResetExpiryScheduler {

    private final PasswordResetRepository repository;

    public PasswordResetExpiryScheduler>PasswordResetRepository repository) {
        this.repository = repository;
    }

    @Scheduled(fixedDelay = 60_000)
    @Transactional
    public void sweepExpiredTokens() {
        List>PasswordResetRequest> active = repository.findByStateIn(
            List.of(
                PasswordResetState.CREATED,
                PasswordResetState.DISPATCHED,
                PasswordResetState.VALIDATED
            )
        );

        active.forEach(req -> {
            req.expire();
            repository.save(req);
        });
    }
}

```

### 3.3.7 Integrasi Service Layer

Keseluruhan state machine dirangkai di dalam `PasswordResetService`. Yang perlu diperhatikan secara khusus adalah bagaimana destination email selalu diambil dari `User.getVerifiedPrimaryEmail()` bukan dari parameter request. Ini berarti meskipun penyerang mengirimkan payload `{"user":{"email":["korban@example.com","hacker@evil.com"]}}`, sistem akan mengabaikannya sepenuhnya karena alamat tujuan sudah terkunci di dalam domain object sejak konstruksi.

```

@Service
@RequiredArgsConstructor
public class PasswordResetService {

    private final UserRepository      userRepository;
    private final PasswordResetRepository resetRepository;
    private final EmailService        emailService;
    private final PasswordEncoder      passwordEncoder;

    @Transactional
    public void initiateReset(String inputEmail) {
        Optional<User> optUser = userRepository
            .findByPrimaryEmail(inputEmail.trim().toLowerCase());

        if (optUser.isEmpty()) return;

        User user = optUser.get();

        VerifiedPrimaryEmail dest = user.getVerifiedPrimaryEmail();

        PasswordResetRequest req = new PasswordResetRequest(dest);
        req.dispatch();

        resetRepository.save(req);
        emailService.sendResetLink(dest.getValue(), req.getTokenHash());
    }

    @Transactional
    public boolean executeReset(String token,
                               String claimedEmail,
                               String newPassword) {
        PasswordResetRequest req = resetRepository
            .findByTokenHash(token)
            .orElse(null);

        if (req == null) return false;

        req.validate(token, claimedEmail);

        req.consume();
        resetRepository.save(req);

        User user = userRepository
            .findByPrimaryEmail(claimedEmail).orElseThrow();
        user.setPassword(passwordEncoder.encode(newPassword));
        userRepository.save(user);

        return true;
    }
}

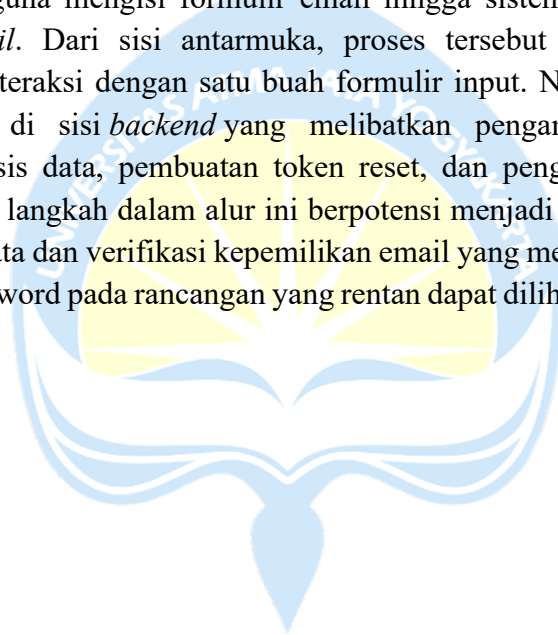
```

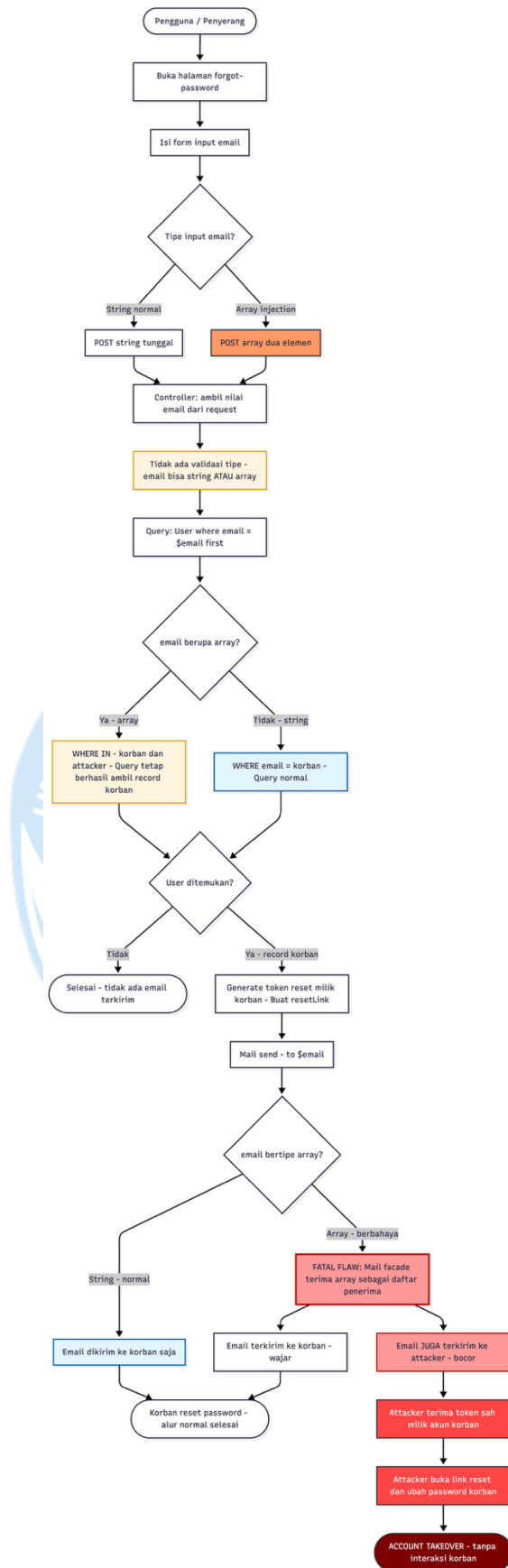
## 4 Mini-Project: Forgot Password: Secure Design Review

Sistem yang dikembangkan pada tahap ini merupakan fitur lupa password berbasis website menggunakan framework Laravel. Dari sisi antarmuka, pengguna hanya melihat satu form input email untuk meminta tautan reset password. Namun pada sisi API, sistem yang dirancang secara rentan tidak membatasi tipe data dari field email, sehingga memungkinkan penyerang mengirimkan email dalam bentuk array melalui payload seperti `user[email][]=korban@example.com&user[email][]=hacker@evil.com`. Kasus ini berkorelasi langsung dengan tema utama CVE-2023-7028 pada GitLab, yang juga berpusat pada manipulasi input identitas akun pada alur reset password untuk mengalihkan token ke email yang tidak sah.

### 4.1 Alur Kerja Sistem

Pada mini-project ini, alur kerja fitur *forgot password* dianalisis dari sudut pandang end-to-end, mulai dari pengguna mengisi formulir email hingga sistem mengirimkan tautan *reset password* melalui *email*. Dari sisi antarmuka, proses tersebut tampak sederhana karena pengguna hanya berinteraksi dengan satu buah formulir input. Namun di balik itu, terdapat serangkaian langkah di sisi *backend* yang melibatkan pengambilan input dari *request*, pencarian akun di basis data, pembuatan token reset, dan pengiriman email melalui *Mail facade* Laravel. Setiap langkah dalam alur ini berpotensi menjadi titik serangan apabila tidak disertai validasi tipe data dan verifikasi kepemilikan email yang memadai. Urutan lengkap alur bisnis sistem lupa password pada rancangan yang rentan dapat dilihat pada Gambar 4.1 berikut.





Gambar 4.1 Alur bisnis system lupa password

Pada rancangan yang rentan, controller menerima input email langsung dari request tanpa memeriksa tipe data yang masuk. Pengambilan data dilakukan dengan `$request->input('user')['email']`, yang dalam PHP akan menerima nilai array tanpa menghasilkan error apapun. Nilai tersebut kemudian digunakan pada dua titik berbahaya secara bersamaan: pertama pada query `User::where('email', $email)`, dan kedua sebagai parameter tujuan pada `$message->to($email)` dalam Mail facade.

Pada titik pertama, Query Builder Laravel secara otomatis mengubah array menjadi klausa `WHERE IN (...)`, sehingga query tetap berhasil mengambil data korban dari database. Pada titik kedua, Mail facade menerima array sebagai tujuan pengiriman, sehingga email reset password dikirimkan ke seluruh alamat dalam array tersebut, termasuk email milik penyerang. Dengan demikian, penyerang menerima token reset yang sah milik akun korban tanpa sepengetahuan korban.

## 4.2 Vulnerable System

Rancangan awal fitur *forgot password* pada mini-project ini secara sengaja mempertahankan kelemahan yang serupa dengan CVE-2023-7028 untuk memperlihatkan bagaimana serangan dapat terjadi dari ujung ke ujung. Dari sisi antarmuka, pengguna hanya berinteraksi dengan satu halaman sederhana berisi kolom input email dan tombol untuk mengirim tautan *reset password*. Di sisi lain, pemrosesan *request* di *backend* tidak membatasi tipe data pada *field* `user[email]`, sehingga *array* email yang dikirim penyerang tetap diterima dan diproses sebagai input yang sah. Tampilan antarmuka sistem lupa password yang rentan ditunjukkan pada Gambar 4.2.



Gambar 4.2 Tampilan antarmuka website lupa password yang rentan

### 4.2.1 Kode Sistem

Struktur kode pada *view* dan *controller* Laravel menunjukkan dengan jelas bagaimana celah keamanan muncul. Berkas `resources/views/forgot-password.blade.php` mendefinisikan *form* dengan nama input `user[email]`, yang oleh PHP dapat diurai menjadi *array* ketika parameter dikirim lebih dari satu kali. Di sisi *backend*, kelas `ForgotPasswordController` pada `app/Http/Controllers/ForgotPasswordController.php` mengambil `$email` dari `$request->input('user')['email']` dan menggunakannya secara langsung baik pada query `User::where('email', $email)->first()` maupun sebagai argumen `$message->to($email)` dalam Mail facade. Pola ini membuat Laravel menyusun query `WHERE IN`

(...) terhadap *array* email dan sekaligus mengirim tautan *reset password* ke seluruh alamat di dalam *array*, sehingga token sah milik korban juga terkirim ke email penyerang.

#### resources/views/forgot-password.blade.php

```
<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">
    <title>Lupa Password</title>
</head>

<body>
    <h1>Lupa Password</h1>
    <form method="POST" action="/forgot-password">
        @csrf
        <label>Email</label>
        <input type="email" name="user[email]" placeholder="Masukkan email Anda"
    />
        <button type="submit">Kirim Tautan Reset</button>
    </form>
</body>
```

#### app/Http/Controllers/ForgotPasswordController.php (Vulnerable)

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Password;

use App\Models\User;
use Illuminate\Support\Facades\Mail;

class ForgotPasswordController extends Controller
{
    public function sendResetLink(Request $request)
    {
        $data = $request->input('user');

        $email = $data['email'];

        $user = User::where('email', $email)->first();

        if ($user) {
            $token = Password::broker()->createToken($user);
            $resetLink = route('password.reset', ['token' => $token], true);
        }
    }
}
```

```

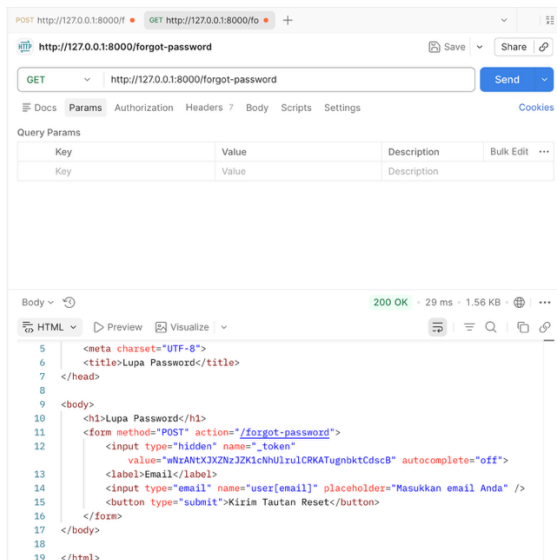
        Mail::send([], [], function ($message) use ($email, $resetLink) {
            $message->to($email)
                ->subject('Tautan Reset Password Anda')
                ->html("Silakan klik link berikut untuk mereset sandi Anda:
{$resetLink}");
        });
    }

    return back()->with('status', 'Link reset telah dikirim');
}
}

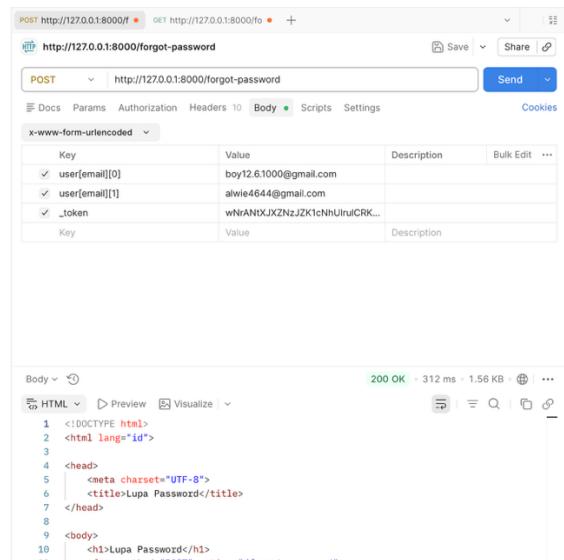
```

Terdapat dua celah kerentanan yang saling memperkuat satu sama lain dalam kode di atas. Berikut adalah skenario eksploitasi lengkapnya:

1. Penyerang mengirim POST request ke /forgot-password dengan payload `user[email][]=korban@gmail.com&user[email][]=attacker@gmail.com`. PHP mengurai field tersebut sebagai array dua elemen.
2. Variabel `$email` berisi `['korban@gmail.com', 'attacker@gmail.com']`. Query `User::where('email', $email)->first()` dieksekusi sebagai `SELECT * FROM users WHERE email IN ('korban@gmail.com', 'attacker@gmail.com') LIMIT 1`. Query berhasil mengembalikan record akun korban.
3. Sistem menghasilkan token reset yang sah untuk akun korban melalui `Password::broker()->createToken($user)` dan membentuk URL tautan reset.
4. `Mail::send()` dipanggil dengan `$message->to($email)` di mana `$email` masih berupa array. Mail facade Laravel menerima array sebagai daftar penerima, sehingga tautan reset yang valid dikirimkan ke `korban@gmail.com` DAN `attacker@gmail.com` secara bersamaan.
5. Penyerang membuka inbox `attacker@gmail.com`, menerima tautan reset, dan menggunakannya untuk mengubah password akun korban. Account takeover berhasil tanpa interaksi apapun dari korban.



Gambar 4.3 Penyerang pergi ke endpoint GET /forgot-password untuk mengambil token csrf



Gambar 4.4 Penyerang mengirim payload array pada field email dan token csrf untuk mengelabui keamanan laravel



Gambar 4.5 Token yang dikirim ke email korban



Gambar 4.6 Token yang dikirim ke email penyerang

## 4.3 Secure System

Perbaikan diterapkan secara berlapis mengikuti prinsip 'make invalid state unrepresentable'. Lapisan pertama adalah domain primitive UserEmail yang menolak tipe non-string sejak object dibuat. Lapisan kedua adalah Form Request yang menolak struktur array di level HTTP. Lapisan ketiga adalah state machine PasswordResetProcess yang menegakkan urutan transisi CREATED → VALIDATED → TOKEN\_SENT → TOKEN\_USED → COMPLETED sehingga tidak ada tahapan yang bisa dilompati. Lapisan keempat adalah pengambilan email tujuan dari record database, bukan dari input pengguna, sehingga Mail facade tidak pernah menerima nilai yang belum terverifikasi.

### 4.3.1 Tambahkan Domain Primitive

Kelas UserEmail dibuat sebagai domain primitive yang memastikan email tidak bisa direpresentasikan sebagai tipe selain string yang lolos validasi format. Seluruh aturan domain ditempatkan di dalam konstruktor sehingga objek yang tidak valid secara struktural tidak mungkin untuk diinstansiasi.

**app/Domain/UserEmail.php**

```
<?php

namespace App\Domain;

use InvalidArgumentException;

final class UserEmail
{
    public readonly string $value;

    public function __construct(string $email)
    {
        $email = trim($email);

        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException(
                'Email tidak valid: format tidak dikenali.'
            );
        }

        $this->value = strtolower($email);
    }

    public function equals(self $other): bool
    {
        return $this->value === $other->value;
    }
}
```

### 4.3.2 Tambahkan Form Request Validation

Form Request dibuat sebagai lapisan penjaga di level HTTP, sebelum controller dieksekusi. Aturan 'string' secara eksplisit menolak struktur array seperti `user[email][]`, sedangkan aturan 'email' menolak format yang tidak valid. Dengan demikian, payload berbahaya ditolak dengan HTTP 422 sebelum logika bisnis apapun berjalan.

#### **app/Http/Requests/ForgotPasswordRequest.php**

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class ForgotPasswordRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'user.email' => ['required', 'string', 'email', 'max:255'],
        ];
    }

    public function messages(): array
    {
        return [
            'user.email.required' => 'Email wajib diisi.',
            'user.email.string' => 'Email harus berupa teks tunggal, bukan
array.',
            'user.email.email' => 'Format email tidak valid.',
        ];
    }
}
```

### 4.3.3 Tambahkan Enum State

Enum `ResetState` mendefinisikan seluruh state yang mungkin pada siklus hidup sebuah permintaan reset password beserta pasangan transisi yang diizinkan melalui method `canTransitionTo()`. State tidak dapat diisi dengan nilai bebas di luar yang terdaftar dalam enum, menghilangkan kemungkinan state tidak terdefinisi.

#### **app/Domain/ResetState.php**

```
<?php
```

```

namespace App\Domain;

enum ResetState: string
{
    case CREATED = 'CREATED';
    case VALIDATED = 'VALIDATED';
    case TOKEN_SENT = 'TOKEN_SENT';
    case TOKEN_USED = 'TOKEN_USED';
    case COMPLETED = 'COMPLETED';
    case REJECTED = 'REJECTED';
    case EXPIRED = 'EXPIRED';

    public function canTransitionTo(self $next): bool
    {
        return match ($this) {
            self::CREATED => in_array($next, [self::VALIDATED, self::REJECTED]),
            self::VALIDATED => in_array($next, [self::TOKEN_SENT,
            self::REJECTED]),
            self::TOKEN_SENT => in_array($next, [self::TOKEN_USED,
            self::EXPIRED]),
            self::TOKEN_USED => $next === self::COMPLETED,
            default => false,
        };
    }

    public function isTerminal(): bool
    {
        return in_array($this, [self::COMPLETED, self::REJECTED, self::EXPIRED]);
    }
}

```

#### 4.3.4 Tambahkan State Machine Domain dan Temporal Invariant

PasswordResetProcess mengintegrasikan seluruh invariant dalam satu domain object. Setiap method memeriksa state saat ini sebelum melakukan operasi apapun. Selain itu, email tujuan pengiriman diambil dari property \$user->email yang berasal dari database, bukan dari input pengguna, sehingga Mail facade tidak pernah menerima nilai yang berpotensi berupa array.

##### 4.3.4.1 Tambahkan Initial State di Konstruktor

Konstruktor PasswordResetProcess memastikan bahwa setiap proses *reset password* selalu dimulai dari *state* yang sama dan batas waktu yang terdefinisi dengan jelas. Saat objek dibuat, *state* awal secara eksplisit diinisialisasi ke `ResetState::CREATED`, waktu pembuatan (`createdAt`) dicatat, dan waktu kedaluwarsa (`expiresAt`) ditetapkan 15 menit sejak saat pembuatan. Dengan cara ini, tidak ada proses *reset* yang lahir tanpa *state* awal yang

valid atau tanpa batas waktu kedaluwarsa, sehingga *temporal invariant* mengenai masa berlaku sesi *reset* ditegakkan sejak awal siklus hidup objek.

```
<?php

namespace App\Domain;

use App\Models\User;
use Carbon\Carbon;
use LogicException;
use Illuminate\Support\Facades\Mail;

class PasswordResetProcess
{
    private ResetState $state;
    private Carbon $createdAt;
    private Carbon $expiresAt;

    public function __construct(
        private readonly UserEmail $inputEmail
    ) {
        $this->state = ResetState::CREATED;
        $this->createdAt = Carbon::now();
        $this->expiresAt = $this->createdAt->copy()->addMinutes(15);
    }

    private function transitionTo(ResetState $next): void
    {
        if (!$this->state->canTransitionTo($next)) {
            throw new LogicException(
                "Transisi tidak valid: {$this->state->value} -> {$next->value}"
            );
        }
        $this->state = $next;
    }

    public function getState(): ResetState
    {
        return $this->state;
    }
}
```

#### 4.3.4.2 Tambahkan Pengecekan State di validate()

Metode `validate()` bertugas memeriksa apakah email yang dimasukkan pengguna benar-benar terhubung dengan akun yang sah sebelum sistem melanjutkan ke tahap pengiriman token. Pengecekan pertama memastikan bahwa `validate()` hanya boleh dipanggil dari *state* `CREATED`, pemanggilan dari *state* lain langsung ditolak karena melanggar urutan proses yang telah didefinisikan. Jika hasil pencarian pengguna menghasilkan `null`, proses tidak dilanjutkan dan *state* diubah menjadi `REJECTED`, sehingga tidak ada token yang akan

dibuat atau dikirim untuk input yang tidak valid. Sebaliknya, jika akun ditemukan, *state* berpindah ke `VALIDATED`, menandai bahwa identitas pengguna telah diverifikasi dan proses siap melanjutkan ke tahap pengiriman token.

```
public function validate(?User $user): void
{
    if ($this->state !== ResetState::CREATED) {
        throw new LogicException(
            "validate() harus dipanggil dari state CREATED, "
            . "bukan {$this->state->value}"
        );
    }

    if ($user === null) {
        $this->transitionTo(ResetState::REJECTED);
        return;
    }

    $this->transitionTo(ResetState::VALIDATED);
}
```

#### 4.3.4.3 Tambahkan Pengecekan State di `sendToken()`

Perbaikan paling kritis terhadap kerentanan asli ada di method ini. Email tujuan pengiriman diambil dari `$user->email` yang merupakan data terverifikasi dari database, bukan dari `$inputEmail` yang berasal dari request pengguna. Dengan demikian, walaupun penyerang memasukkan array sebagai input, Mail facade hanya akan menerima string tunggal yang sudah divalidasi.

```
public function sendToken(User $user, string $resetLink): void
{
    if ($this->state !== ResetState::VALIDATED) {
        throw new LogicException(
            "sendToken() hanya valid dari state VALIDATED, "
            . "bukan {$this->state->value}"
        );
    }

    if (Carbon::now()->greaterThan($this->expiresAt)) {
        $this->transitionTo(ResetState::EXPIRED);
        throw new LogicException('Sesi reset sudah kedaluwarsa.');
```

```
    }

    Mail::send([], [], function ($message) use ($user, $resetLink) {
        $message->to($user->email)
            ->subject('Tautan Reset Password Anda')
            ->html(
                "Silakan klik link berikut untuk mereset sandi Anda: "
                . "{$resetLink}"
            );
    });
}
```

```

        );
    });

    $this->transitionTo(ResetState::TOKEN_SENT);
}

```

#### 4.3.4.4 Tambahkan Pengecekan State di useToken()

Metode `useToken()` merepresentasikan momen ketika pengguna mengklik tautan *reset password* dan sistem mencatat bahwa token sedang digunakan. Untuk menjaga konsistensi alur, metode ini hanya boleh dipanggil ketika *state* berada pada `TOKEN_SENT`; setiap pemanggilan dari *state* lain akan langsung menghasilkan *exception* karena melanggar *temporal invariant* yang telah ditetapkan. Sebelum *state* berpindah ke `TOKEN_USED`, sistem kembali memeriksa apakah waktu kedaluwarsa telah terlewati. Jika token sudah kedaluwarsa, *state* diubah menjadi `EXPIRED` dan eksekusi dihentikan, sehingga token yang sudah basi tidak pernah dapat digunakan lagi. Dengan cara ini, penggunaan ulang token di luar jendela waktu yang sah dapat dicegah secara struktural.

```

public function useToken(): void
{
    if ($this->state !== ResetState::TOKEN_SENT) {
        throw new LogicException(
            "useToken() hanya valid dari state TOKEN_SENT, "
            . "bukan {$this->state->value}"
        );
    }

    if (Carbon::now()->greaterThan($this->expiresAt)) {
        $this->transitionTo(ResetState::EXPIRED);
        throw new LogicException('Token reset sudah kedaluwarsa.');
```

```

        $this->transitionTo(ResetState::TOKEN_USED);
    }
}

```

#### 4.3.4.5 Tambahkan Pengecekan State di complete()

Metode `complete()` menutup seluruh siklus hidup proses *reset password* dengan menandai bahwa token telah digunakan dan perubahan kata sandi telah selesai diterapkan. Agar alur tidak dapat dilompati, metode ini hanya boleh dipanggil dari *state* `TOKEN_USED`; jika dipanggil dari *state* lain, sistem segera melempar *exception* karena urutan operasi tidak sesuai dengan kontrak yang ditetapkan. Setelah *state* berpindah ke `COMPLETED`, proses *reset* dianggap final dan tidak ada lagi operasi lanjutan yang diperbolehkan pada objek yang sama. Pola ini memastikan bahwa setiap token hanya melewati urutan `CREATED` → `VALIDATED` → `TOKEN_SENT` → `TOKEN_USED` → `COMPLETED` tanpa bisa melompat atau kembali mundur.

```

public function complete(): void
{
    if ($this->state !== ResetState::TOKEN_USED) {
        throw new LogicException(
            "complete() hanya valid dari state TOKEN_USED, "
            . "bukan {$this->state->value}"
        );
    }

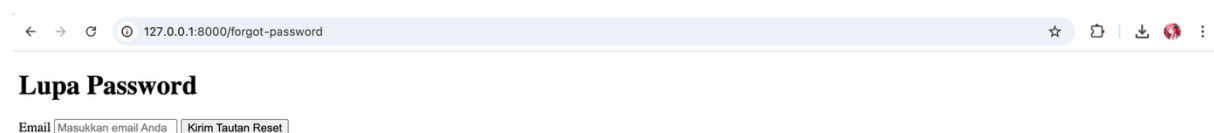
    $this->transitionTo(ResetState::COMPLETED);
}

```

## 4.4 Kode Final Secure Design

Pada tahap akhir, seluruh konsep *domain primitive*, *state machine*, dan *temporal invariant* digabungkan ke dalam satu rancangan *controller* yang aman untuk fitur *forgot password*. Antarmuka pengguna tetap dipertahankan sederhana seperti pada sistem yang rentan, sehingga dari sudut pandang pengguna tidak ada perbedaan perilaku yang mencolok antara versi lama dan versi yang telah diperkuat. Perbedaan utama justru berada pada lapisan *backend*, di mana setiap input email melewati beberapa lapis validasi mulai dari *ForgotPasswordRequest*, *value object* *UserEmail*, hingga objek *domain* *PasswordResetProcess* yang mengontrol alur *state*.

Pada implementasi ini, *controller* tidak lagi memproses *array* email secara langsung, melainkan selalu bekerja dengan representasi email yang sudah tervalidasi dan terikat ke akun pengguna yang sah. Setiap langkah kritis seperti validasi pengguna, pembuatan token, pengiriman email, dan pencatatan kesalahan dijalankan sesuai urutan *state* yang diizinkan oleh *ResetState*. Apabila terjadi pelanggaran urutan atau kondisi yang tidak valid, proses segera dihentikan dan dicatat di *log* tanpa mengungkapkan informasi sensitif kepada penyerang. Tampilan antarmuka dan cuplikan kode lengkap *ForgotPasswordController* yang telah menerapkan desain aman ini dapat dilihat pada Gambar 4.7 dan listing kode berikut.



Gambar 4.7 Tampilan antarmuka website lupa password yang secure

## app/Http/Controllers/ForgotPasswordController.php (Secure)

```
<?php

namespace App\Http\Controllers;

use App\Domain>PasswordResetProcess;
use App\Domain\UserEmail;
use App\Http\Requests\ForgotPasswordRequest;
use App\Models\User;
use Illuminate\Support\Facades\Password;
use InvalidArgumentException;
use LogicException;

class ForgotPasswordController extends Controller
{
    public function sendResetLink(ForgotPasswordRequest $request)
    {
        $rawEmail = $request->validated()['user']['email'];

        try {
            $email = new UserEmail($rawEmail);
        } catch (InvalidArgumentException $e) {
            return back()->withErrors(['user.email' => $e->getMessage()]);
        }

        $process = new PasswordResetProcess($email);

        try {
            $user = User::where('email', $email->value)->first();

            $process->validate($user);

            if ($process->getState() === \App\Domain\ResetState::VALIDATED) {
                $token = Password::broker()->createToken($user);
                $resetLink = route('password.reset', ['token' => $token], true);

                $process->sendToken($user, $resetLink);
            }

        } catch (LogicException $e) {
            logger()->warning('[ForgotPassword] State machine error: ' . $e->getMessage(), [
                'input_email' => $rawEmail,
            ]);
        }

        return back()->with(
            'status',
            'Jika email Anda terdaftar, tautan reset akan segera dikirim.'
        );
    }
}
```

```

    };
  }
}

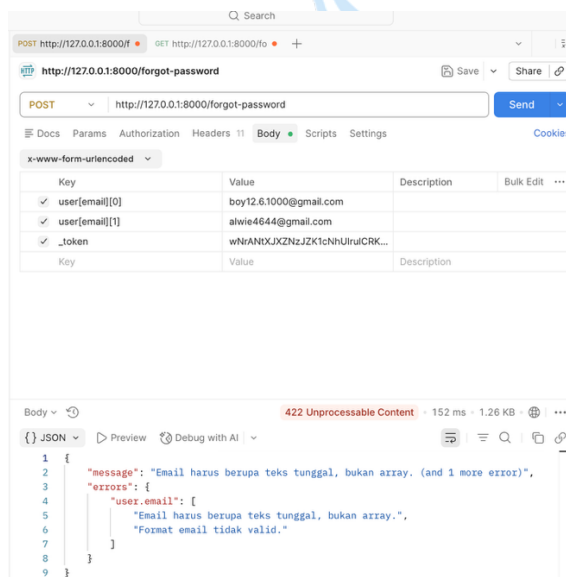
```

## 4.5 Simulasi Serangan pada Secure System

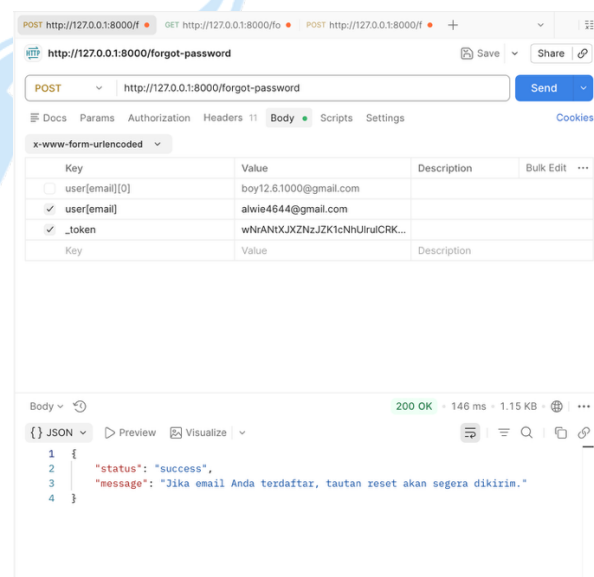
Ketika penyerang mengirimkan payload `user[email][]=korban@gmail.com&user[email][]=attacker@gmail.com`, Form Request ForgotPasswordRequest mendeteksi bahwa nilai `user.email` bukan string melainkan array. Aturan validasi 'string' langsung gagal dan request ditolak dengan HTTP 422 sebelum menyentuh controller sama sekali. Tidak ada query database yang dieksekusi dan tidak ada token yang dibuat.

Apabila penyerang berhasil melewati Form Request dengan cara lain, lapisan kedua aktif: konstruktor `UserEmail` menerima parameter bertipe string (PHP type-hint), sehingga array akan menyebabkan `TypeError` sebelum objek terbentuk. Apabila input lolos validasi tetapi email tidak ditemukan di database, domain object bertransisi ke state `REJECTED` melalui `validate()`, dan pengiriman email tidak pernah terjadi.

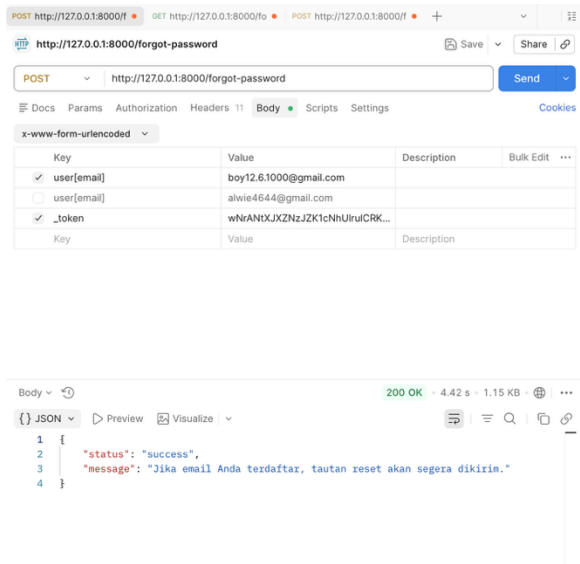
Yang paling krusial: walaupun semua lapisan di atas dilewati, email tujuan pada `Mail::send()` tidak pernah menggunakan nilai dari input pengguna. Nilai yang digunakan adalah `$user->email` yang merupakan string tunggal dari database. Dengan demikian, penyerang tidak dapat menyisipkan alamat emailnya sendiri sebagai penerima token reset, menutup secara tuntas celah yang sama dengan CVE-2023-7028.



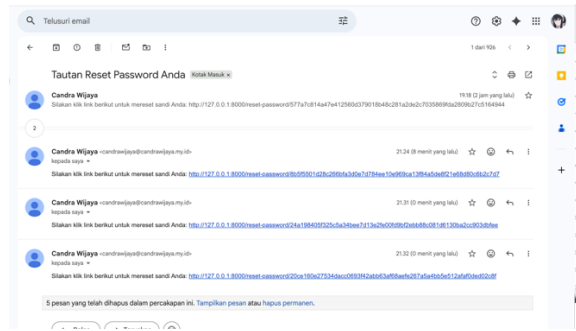
Gambar 4.8 Penyerang mengirim payload array ditolak HTTP 422 oleh Form Request



Gambar 4.9 Respons seragam mencegah penyerang melakukan enumerasi akun (User Tidak Terdaftar)



Gambar 4.10 Respons seragam mencegah penyerang melakukan enumerasi akun (User Terdaftar)



Gambar 4.11 Token akan dikirim apabila User Valid (User Terdaftar)



## DAFTAR PUSTAKA

- [1] National Institute of Standards and Technology (NIST), "CVE-2023-7028 Detail," National Vulnerability Database (NVD), Published Jan. 12, 2024, Last Modified Oct. 24, 2025. [Online]. Available: <https://nvd.nist.gov/vuln/detail/cve-2023-7028>
- [2] GitLab Inc., "Critical Security Release: GitLab 16.7.2 Released — CVE-2023-7028 Account Takeover via Password Reset," GitLab Security Advisory, Jan. 11, 2024. [Online]. Available: <https://about.gitlab.com/releases/2024/01/11/critical-security-release-gitlab-16-7-2-released/>
- [3] Vozec, "CVE-2023-7028: Account-Take-Over GitLab — Proof of Concept," GitHub Repository, 2024. [Online]. Available: <https://github.com/Vozec/CVE-2023-7028>
- [4] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2003.
- [5] OWASP Foundation, "OWASP Top 10: The Ten Most Critical Web Application Security Risks," 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [6] Laravel, "Resetting Passwords," Laravel 13.x Documentation. [Online]. Available: <https://laravel.com/docs/13.x/passwords>
- [7] CloudSEK, "Exposed: How a Single API Flaw Put Millions of Medical Records at Risk," 2024. [Online]. Available: <https://www.cloudsek.com/blog/exposed-how-a-single-api-flaw-put-millions-of-medical-records-at-risk>
- [8] Laravel, "Mail," Laravel 13.x Documentation. [Online]. Available: <https://laravel.com/docs/13.x/mail>
- [9] PHP, "filter\_var," PHP Manual. [Online]. Available: <https://www.php.net/manual/en/function.filter-var.php>